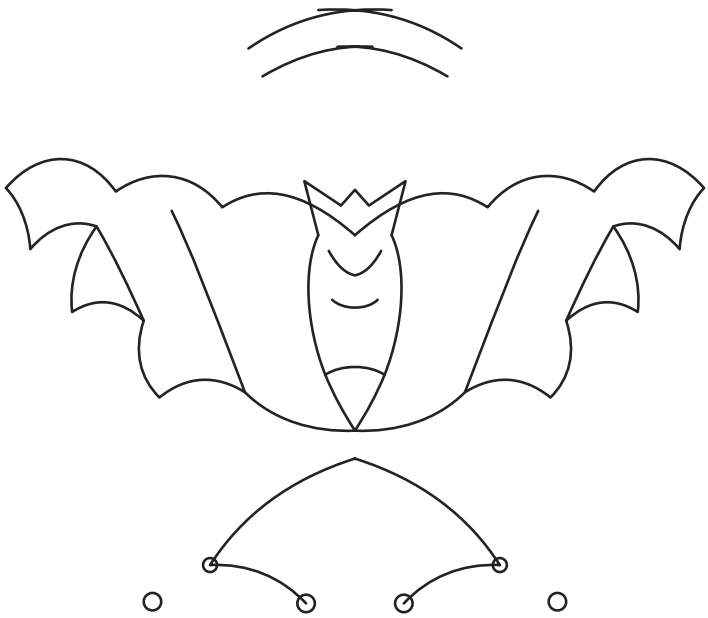
# Del texto al contexto

Tokens, vectores y búsqueda semántica paso a paso



ESCRITO E ILUSTRADO POR

Biblioteca de Agentes de IA

XI
MURCIÉLAGO

# Del texto al contexto

## Tokens, vectores y búsqueda semántica paso a paso

Biblioteca de Agentes de IA · XI
2026

## Sobre la portada

Un murciélago orienta su vuelo al recibir ecos. Este libro sigue otra señal: la semejanza entre representaciones numéricas de textos.

La silueta de la cubierta se dibuja entre ondas y puntos. Las ondas sugieren una consulta; los puntos, posibles fragmentos de una colección. La imagen invita a mirar el recorrido completo, desde la pregunta hasta la fuente que se recupera.

La metáfora termina antes de la decisión. Un eco físico informa sobre la posición de un objeto; una puntuación vectorial solo mide semejanza según un modelo y una métrica. No sabe si una política está vigente, si el lector tiene permiso ni si el texto dice la verdad. Esas comprobaciones pertenecen al sistema y a las personas.

# Prefacio

## ¿Cómo pasa una pregunta escrita a una respuesta que cita un documento concreto?

La frase «la IA busca en una base vectorial» esconde operaciones distintas. Primero se preparan los documentos. Después se convierte la pregunta en un vector, se recuperan candidatos y se verifica cuál puede usarse. Solo entonces un modelo redacta. La tokenización interviene en varios puntos, pero no es la búsqueda misma.

Seguiremos una consulta ficticia y pequeña: «¿Cómo recupero el acceso al Portal de Proveedores?». La colección de ejemplo contiene una guía vigente, una guía retirada, una pregunta frecuente y un texto de facturación. Los números de los gráficos se inventaron para enseñar las operaciones; no son embeddings calculados por un servicio real.

Este volumen amplía la explicación de tokens del volumen X y se conecta con la organización documental de los volúmenes VIII y IX. Está escrito para lectores que quieren entender qué hace cada etapa antes de implementar un sistema de recuperación aumentada.

## Cómo leer este libro

Las tres partes siguen el orden de una consulta real: preparar, recuperar y responder. Puede leerse de principio a fin sin programar. Quien ya conoce tokens puede comenzar en el capítulo 2; quien diseña sistemas debería detenerse también en los capítulos 4 y 5.

Las figuras usan la misma pregunta y cuatro fragmentos ficticios. En los gráficos bidimensionales, cada vector tiene solo dos coordenadas para poder dibujarse. Los embeddings de producción suelen tener muchas más dimensiones y no poseen ejes interpretables tan cómodos.

**RECUERDA**

Convención visual: morado para consulta y proceso; verde para fuente vigente; gris para contexto auxiliar; terracota para un documento retirado. Un número alto en el gráfico indica parecido, no autoridad.

# Índice

PARTE I · PREPARAR LA INFORMACIÓN

1 El texto se convierte en piezas

Tres cosas que no son sinónimos

2 Preparar la biblioteca antes de buscar

Qué hace la base vectorial

PARTE II · ENCONTRAR FRAGMENTOS

3 El coseno: comparar direcciones

4 De los vecinos a la evidencia

La búsqueda puede ser mixta

PARTE III · CONSTRUIR UNA RESPUESTA

5 Del fragmento a la respuesta

Una prueba mínima de calidad

6 Un laboratorio que cabe en una página

Pruebe tres cambios

CIERRE

Glosario

Fuentes

Colofón

PARTE I

# Preparar la información

Antes de buscar, el sistema convierte documentos en fragmentos identificables y representaciones numéricas.

CAPÍTULO 1

# El texto se convierte en piezas

> *Antes de calcular semejanzas, conviene saber qué objetos entran y salen de cada etapa.*

Una persona pregunta: «¿Cómo recupero el acceso al Portal de Proveedores?». Para nosotros es una oración. Para un modelo de lenguaje, primero se convierte en una secuencia de unidades llamadas *tokens*. Un token puede ser una palabra completa, un signo o una parte de palabra. Depende del tokenizador y de su vocabulario; no existe una separación universal para esa oración.

DEFINICIÓN

**Tokenización.** Conversión de texto en unidades de un vocabulario, normalmente representadas por identificadores numéricos. Los modelos modernos suelen usar subpalabras para combinar un vocabulario manejable con la posibilidad de representar términos poco frecuentes.

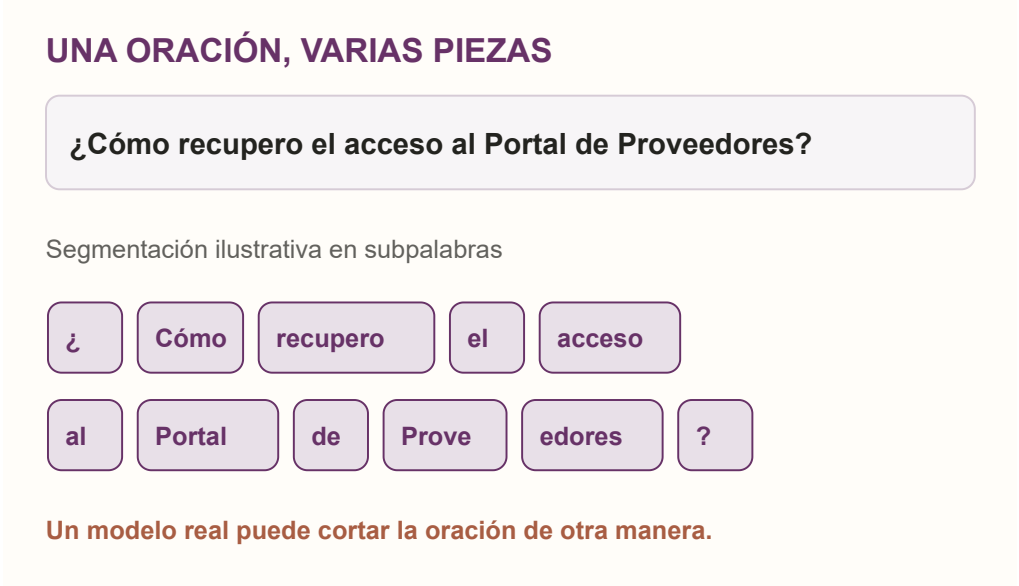



**Figura 1.1** · Una división didáctica de la pregunta en piezas. La separación real depende del tokenizador.

La ilustración divide «proveedores» en «prove» y «edores» solo para mostrar la idea de subpalabra. No afirma que un modelo concreto lo haga así. La documentación de tokenizadores describe familias como BPE, WordPiece y Unigram; cada una aprende o aplica reglas distintas. Un cambio de tokenizador puede cambiar el número de tokens aunque el texto visible no cambie.

### Tres cosas que no son sinónimos

Un **token** es una pieza que procesa un modelo. Un **fragmento** o *chunk* es una sección de documento que el sistema decide guardar y recuperar, por ejemplo un párrafo con su título. Un **embedding** es una lista de números calculada para representar una entrada. Un fragmento puede ocupar muchos tokens y producir un único embedding para la búsqueda.

Esto resuelve una confusión habitual: la tokenización por sí sola no consulta la base vectorial. El modelo que crea embeddings procesa el texto de la pregunta; luego el buscador compara el vector obtenido con

vectores previamente guardados. Más tarde, el modelo que redacta también tokeniza los fragmentos recibidos en su ventana de contexto. Es posible que ambos modelos utilicen tokenizadores diferentes.

**CASO**

En nuestro ejemplo, la pregunta escrita por la persona llega al sistema de búsqueda. El texto original se conserva para mostrarlo y para la respuesta. Una representación numérica sirve para encontrar candidatos. Nunca se reconstruye la guía de acceso a partir de los números: el sistema guarda o enlaza el texto fuente por separado.

**RECUERDA**

Si ves «tokens → vector → resultado», pregunta siempre: ¿tokens de qué texto?, ¿qué modelo creó el vector? y ¿dónde está el fragmento original?

CAPÍTULO 2

# Preparar la biblioteca antes de buscar

> *Una consulta rápida depende de un trabajo previo y verificable sobre las fuentes.*

Supongamos que el equipo de soporte conserva cuatro textos: la guía de acceso vigente, una versión retirada, una pregunta frecuente y una instrucción de facturación. Antes de responder consultas, el sistema extrae su contenido, identifica título, fecha, versión, propietario y permisos, y los divide en fragmentos recuperables. Esta fase se llama **indexación**.

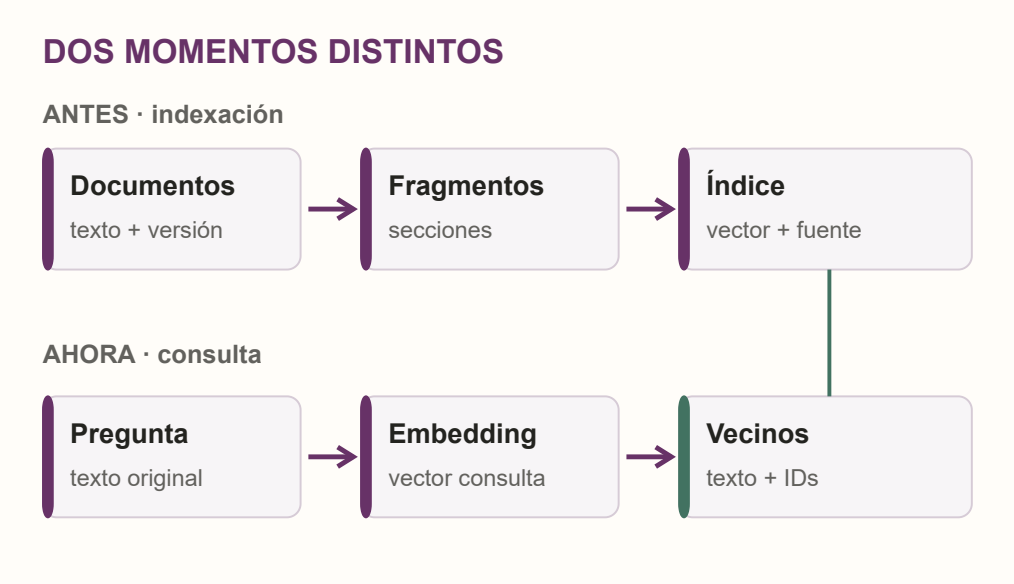



**Figura 2.1** · Dos recorridos separados: preparar documentos una vez y buscar con cada pregunta.

El tamaño del fragmento es una decisión práctica. Si se guarda el manual entero como un solo vector, la respuesta específica sobre acceso puede quedar diluida entre muchas secciones. Si se corta cada línea sin contexto, «pulse aquí» pierde el título y las condiciones. Un

corte útil conserva la unidad de sentido, incluye el encabezado y apunta al documento y sección originales. A veces se repiten unas líneas entre fragmentos vecinos para no partir una explicación.

DEFINICIÓN

**Registro indexado.** Conjunto de texto recuperable, embedding y metadatos. La base vectorial almacena el vector para localizar vecinos; el texto y la referencia permiten leer y citar lo encontrado. Según la arquitectura, texto y vector pueden vivir en la misma base o en sistemas conectados.

| ID | Fragmento de ejemplo | Estado |
|---|---|---|
| A | «Use Olvidé mi contraseña; el enlace dura 20 minutos» | Vigente |
| B | «Solicite el restablecimiento por correo al administrador» | Retirado |
| C | «Si no llega el correo, revise spam y contacte soporte» | Vigente |
| D | «Descargue las facturas desde la sección Pagos» | Vigente |

El modelo de embeddings calcula un vector para cada registro. Durante la consulta debe usarse una representación compatible con la del índice; mezclar modelos o versiones sin reindexar puede volver incomparables las coordenadas. Para mantener el índice útil, un cambio de documento requiere actualizar texto, vector y metadatos; retirar una versión requiere excluirla de futuras respuestas.

## Qué hace la base vectorial

Permite encontrar registros cercanos a un vector de consulta según una distancia elegida. Algunas implementaciones ofrecen búsqueda exacta y aproximada; la segunda puede acelerar corpus grandes con una posible pérdida de resultados cercanos. pgvector documenta

distancia coseno mediante el operador `<=>` y calcula similitud coseno como uno menos esa distancia. Eso describe una implementación, no una regla que obligue a todos los sistemas a usar coseno.

> **ADVERTENCIA**
>
> Indexar un documento no concede permiso para mostrarlo. Los controles de acceso deben aplicarse en la recuperación y verificarse de nuevo antes de construir la respuesta.

> **RECUERDA**
>
> La indexación no guarda «conocimiento» libre de contexto. Guarda representaciones y enlaces a textos con versión, fecha y dueño.

PARTE II

# Encontrar fragmentos

Una consulta recorre el mismo espacio vectorial y produce candidatos ordenados por semejanza.

CAPÍTULO 3

# El coseno: comparar direcciones

> *La búsqueda ordena candidatos; el ángulo explica una forma frecuente de hacerlo.*

La pregunta entra al mismo tipo de modelo de embeddings que se usó al indexar. Sale un vector de consulta. Para poder dibujarlo, imaginemos un mundo de solo dos dimensiones. Nuestra pregunta vale `q = (3, 4)`. La guía vigente vale `A = (4, 3)`. Ambos tienen longitud 5. Son números inventados para la explicación: ningún eje equivale literalmente a «acceso» o «proveedores».

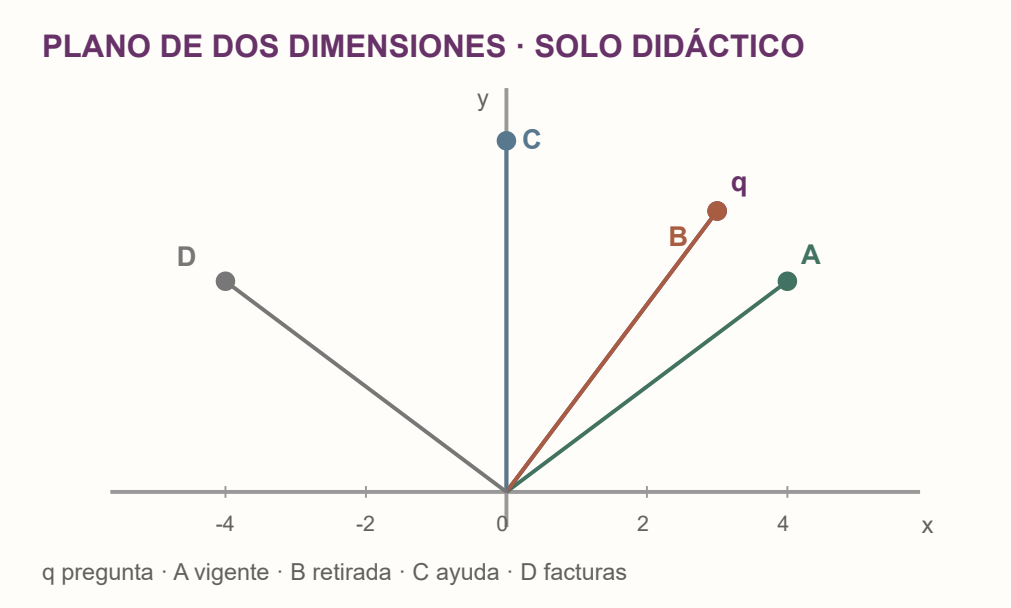



**Figura 3.1** · Vectores didácticos de dos dimensiones. La guía retirada B está en la misma dirección que la consulta, aunque ya no es válida.

La similitud coseno compara el ángulo entre dos vectores. Se calcula como **(q · d) / (||q|| × ||d||)**, donde «·» es el producto escalar y las barras dobles son las longitudes. Para A: `(3×4 + 4×3) / (5×5) = 24/25 =`

`0,96`. La guía retirada B tiene `(3,4)`, de modo que puntúa `1,00`. C, la ayuda sobre correo, tiene `(0,5)` y puntúa `0,80`. D, sobre facturas, tiene `(−4,3)` y puntúa `0,00`.

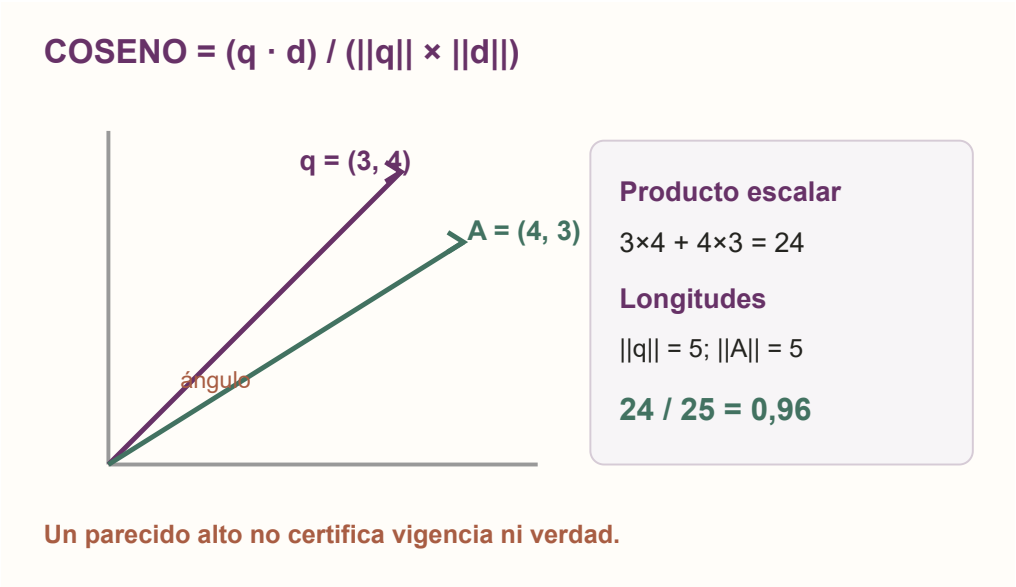



**Figura 3.2** · La longitud de la flecha no decide la similitud coseno; importa su dirección relativa.

DEFINICIÓN

**Similitud coseno.** Medida de alineación entre dos vectores no nulos. Vale 1 si apuntan en la misma dirección, 0 si son perpendiculares y −1 si apuntan en direcciones opuestas. En embeddings reales, un puntaje alto no se traduce automáticamente en un porcentaje de verdad o de relevancia.

En muchas búsquedas se ordenan primero los resultados de mayor similitud. Una base también puede trabajar con *distancia coseno*, donde menor es mejor. La definición concreta y los índices disponibles dependen de la herramienta. Además, hay sistemas que prefieren producto escalar u otras distancias, y otros combinan señales semánticas con términos exactos.

**ADVERTENCIA**

El 1,00 de B es una trampa deliberada: representa una coincidencia perfecta con un procedimiento que ya fue retirado. El coseno no lee la fecha ni decide qué documento tiene autoridad.

**RECUERDA**

El coseno ayuda a responder «¿qué vector se parece?». La pregunta «¿qué fuente puedo usar?» requiere metadatos, permisos y lectura del contenido.

CAPÍTULO 4

# De los vecinos a la evidencia

> *Los candidatos de la base son el comienzo de la lectura, no el final de la decisión.*

Una búsqueda ingenua que pidiera los dos vecinos más próximos devolvería B y A. Si el sistema entregara ambos sin más, el modelo recibiría instrucciones contradictorias: un correo al administrador y un botón de autoservicio. Nuestro corpus contiene el dato que resuelve la contradicción: B está marcado como retirado. La recuperación útil debe considerar esa marca antes de redactar.

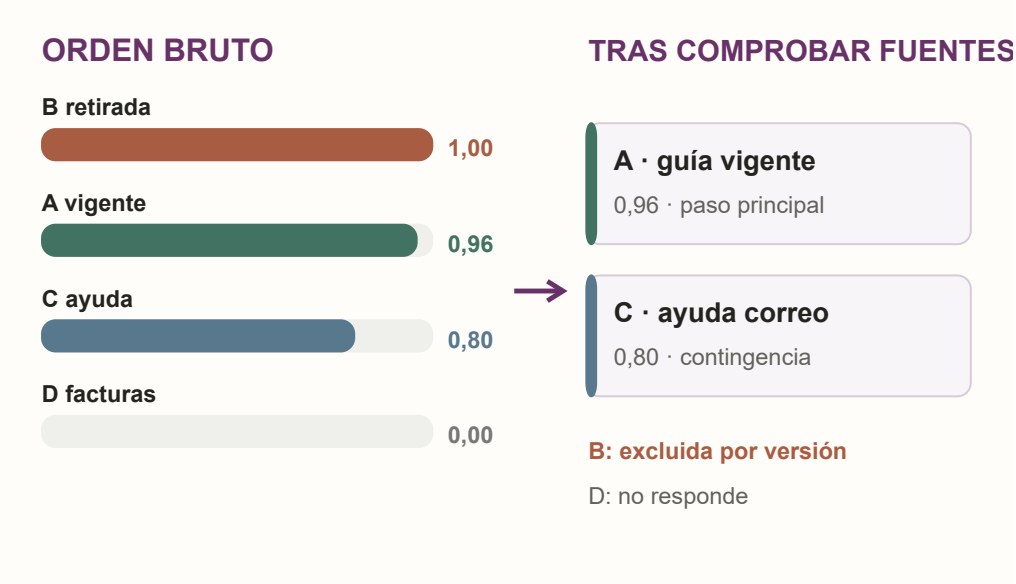



**Figura 4.1** · El orden bruto de similitud y el resultado después de excluir la guía retirada.

El filtro puede aplicarse durante la consulta si la base lo soporta, o mediante una comprobación estricta antes de entregar los textos. En ambos casos hay que evitar que un fragmento sin autorización se cuele en el contexto. Después de filtrar, A queda como guía principal y C como ayuda complementaria. D no responde a la pregunta aunque pertenezca al mismo portal.

### La búsqueda puede ser mixta

La semejanza semántica resulta útil cuando el usuario dice «no puedo entrar» y la guía dice «restablecimiento de credenciales». Sin embargo, un código de error, un número de contrato o el nombre exacto de una función puede beneficiarse de búsqueda por términos. Una arquitectura puede combinar búsqueda textual, filtros por dominio y vectores; incluso puede reordenar los candidatos con otra etapa. La base vectorial es una pieza, no todo el sistema.

También importa la procedencia. Cada resultado debe llevar una ruta estable, sección, versión y fecha. Si dos fragmentos se contradicen, no basta con que uno tenga 0,96 y otro 0,80: se revisa cuál es la fuente autorizada y qué afirma realmente. Si ningún texto vigente contesta, la salida correcta es indicar que falta evidencia.

CASO

Consulta: «¿Cómo recupero el acceso?». B aparece primero por similitud, pero se descarta porque su estado es «retirado». A indica el paso de autoservicio. C explica qué hacer si no llega el correo. El conjunto final contiene A y C con sus referencias; B se conserva solo para auditoría, nunca para instruir al usuario.

DECISIÓN DE ARQUITECTURA

| | |
|---|---|
| **Contexto** | Hay documentos parecidos con distintas fechas y permisos. |
| **Decisión** | Recuperar por similitud dentro del dominio y aplicar versión, vigencia y acceso antes del contexto generativo. |
| **Consecuencia** | El índice necesita metadatos mantenidos y pruebas con versiones retiradas. |

**RECUERDA**

El buscador propone candidatos. La política del sistema y el contenido de la fuente determinan cuáles pueden sostener una respuesta.

PARTE III

# Construir una respuesta

Los fragmentos elegidos se comprueban y pasan al modelo para redactar una respuesta trazable.

CAPÍTULO 5

# Del fragmento a la respuesta

> *La generación ocurre después de recuperar texto; el vector ya cumplió su función de búsqueda.*

El sistema prepara una entrada para el modelo de lenguaje: la pregunta original, una instrucción de responder con la información disponible y los fragmentos A y C con sus referencias. El modelo no recibe una «verdad vectorial». Recibe texto legible dentro de su ventana de contexto. Lo procesa en tokens y genera la respuesta token a token, como se explica en el volumen X.

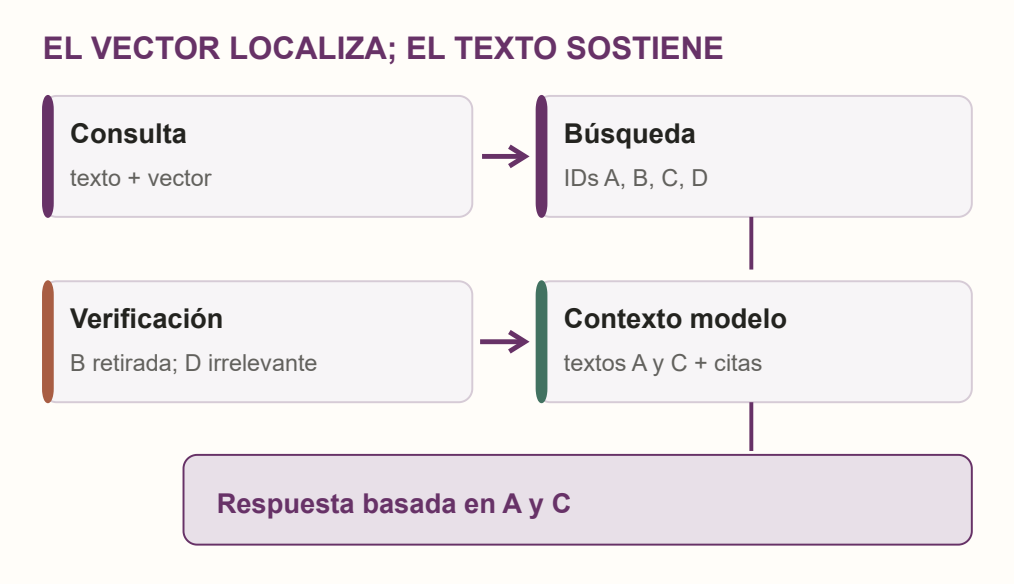



**Figura 5.1** · El vector sirve para localizar; la respuesta se fundamenta en texto y referencias.

Una respuesta razonable para el caso sería: «En el Portal de Proveedores, pulse “Olvidé mi contraseña” y use el enlace recibido dentro de 20 minutos (Guía de acceso, sección 2). Si no llega el correo, revise spam y contacte a soporte (Preguntas frecuentes, sección 4)». Es

un ejemplo construido con documentos ficticios. Las citas identifican el origen de cada afirmación; el sistema debe generar enlaces reales cuando trabaje con fuentes reales.

DEFINICIÓN

**Generación aumentada por recuperación (RAG).** Patrón en el que un sistema aporta al modelo información recuperada de una fuente externa para una tarea de generación. Recuperar texto útil puede mejorar una respuesta, pero la arquitectura necesita controles de selección, vigencia, permisos y verificación.

Hay dos fallos distintos. El primero es recuperar el texto equivocado: un fragmento antiguo o una coincidencia superficial. El segundo es redactar algo que no está sostenido por el texto correcto: una cifra inventada, un paso omitido o una cita que apunta a otra sección. Por eso conviene evaluar la recuperación y la respuesta por separado. Una puntuación alta de búsqueda no certifica la respuesta final.

### Una prueba mínima de calidad

Para cada pregunta de prueba, registre qué fragmentos eran esperados, cuáles llegaron, cuáles se excluyeron y por qué. Luego revise si cada afirmación de la respuesta aparece en las fuentes citadas. En este caso, el sistema pasa si omite la guía B, usa A para el primer paso, usa C solo para la contingencia y reconoce cuando no encuentra un procedimiento vigente.

ADVERTENCIA

El texto recuperado puede contener instrucciones dirigidas al modelo, pero sigue siendo contenido de una fuente. No debe adquirir autoridad para cambiar las reglas del sistema ni para pedir datos o acciones ajenas a la pregunta.

**RECUERDA**

El flujo completo es: pregunta → embedding de consulta → vecinos → filtros y lectura → fragmentos autorizados → modelo generativo → respuesta con citas.

CAPÍTULO 6

# Un laboratorio que cabe en una página

> *Repetir el cálculo a mano revela qué hace la métrica y qué nunca puede decidir.*

Use la tabla del capítulo 2 y estos vectores didácticos: consulta `q=(3,4)`, A vigente `(4,3)`, B retirado `(3,4)`, C vigente `(0,5)` y D vigente `(−4,3)`. Todos tienen longitud 5; por eso cada producto escalar se divide por 25. El resultado es reproducible con lápiz o calculadora.

| Registro | Producto q·d | Coseno | Decisión |
|---|---|---|---|
| B · guía retirada | 25 | 1,00 | Excluir por versión |
| A · guía vigente | 24 | 0,96 | Usar como fuente principal |
| C · ayuda de correo | 20 | 0,80 | Usar si aporta la contingencia |
| D · facturas | 0 | 0,00 | Descartar por tema |

## Pruebe tres cambios

**1. Cambie la pregunta.** «¿Dónde descargo una factura?» exigiría otro vector de consulta. No basta con cambiar las palabras en la pantalla y conservar `(3,4)`. El modelo de embeddings debe volver a codificarla.

**2. Cambie la versión.** Si B se convierte en la guía vigente y A se retira, el cálculo coseno permanece igual. La decisión cambia por los metadatos. Esto muestra por qué la política de fuentes no puede esconderse dentro de la métrica.

**3. Quite la evidencia.** Si A y C no son accesibles para la persona, el sistema no debe componer una respuesta a partir de B o D. Debe explicar que no dispone de una guía autorizada y orientar al canal responsable.

CASO

**Resultado esperado.** El lector puede señalar el tokenizador de cada modelo, el lugar donde se guarda el texto, la operación de coseno, el filtro que elimina B y las frases de A y C que sostienen la respuesta. Si uno de esos pasos no se puede señalar, el diagrama del sistema todavía está incompleto.

El ejercicio es pequeño a propósito. En producción habrá miles o millones de registros, vectores de muchas dimensiones y un índice optimizado. La estructura lógica, sin embargo, sigue siendo reconocible: representar, encontrar, comprobar y responder. Los volúmenes VIII y IX de esta biblioteca profundizan en la organización, autoridad y federación de las fuentes.

RECUERDA

La semejanza es una pista útil para encontrar información. La respuesta fiable surge al unir esa pista con un texto verificable y una decisión explícita sobre qué fuente se puede usar.

CIERRE

# Síntesis

Un token es una unidad de texto para un modelo. Un fragmento es una porción de documento elegida para indexar o leer. Un embedding representa un texto como un vector. Son tres objetos diferentes.

Durante la preparación se extraen y dividen documentos, se guardan texto y metadatos, y se calculan embeddings de los fragmentos. Durante la consulta se calcula el embedding de la pregunta con un sistema compatible y se buscan vecinos. El coseno compara direcciones: producto escalar dividido por las longitudes. La base devuelve candidatos y referencias a sus textos.

La lista debe pasar por permisos, vigencia y comprobación de evidencia. Una guía retirada puede puntuar más alto que la actual. El modelo generativo recibe la pregunta y los fragmentos autorizados dentro de su contexto; produce la respuesta token a token. Una cita debe apuntar al texto de origen, no al número del vector.

REFERENCIA

# Glosario

**Base vectorial**

Sistema que almacena vectores y permite buscar los más cercanos según una distancia o similitud. El diseño también debe conservar o enlazar texto y metadatos.

**Coseno**

Medida de alineación entre vectores no nulos: producto escalar dividido por el producto de sus longitudes.

**Embedding**

Vector numérico calculado por un modelo para representar una entrada, como un fragmento de texto o una pregunta.

**Fragmento**

Porción de documento que el sistema decide indexar y recuperar como unidad, normalmente acompañada de título y referencia.

**Índice**

Estructura que permite localizar registros; en la búsqueda vectorial puede incluir mecanismos exactos o aproximados.

**Metadatos**

Datos sobre una fuente, como título, versión, fecha, propietario, permisos y ubicación.

**RAG**

Generación aumentada por recuperación: patrón que aporta al modelo información recuperada de una fuente externa para generar una respuesta.

**Similitud coseno**

Valor que expresa cuánto coinciden las direcciones de dos vectores; no mide verdad, vigencia ni autorización.

**Token**

Unidad de un vocabulario que procesa un modelo; puede ser palabra, subpalabra, signo u otra pieza según el tokenizador.

**Tokenización**

Proceso que convierte texto en tokens e identificadores según las reglas de un tokenizador.

REFERENCIA

# Fuentes


1. Hugging Face. Tokenization algorithms. https://huggingface.co/docs/transformers/v5.4.0/tokenizer_summary. Consulta: 2026-10-17.
2. pgvector. pgvector README. https://github.com/pgvector/pgvector. Consulta: 2026-10-17.
3. Lewis et al.. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. https://arxiv.org/abs/2005.11401. Consulta: 2026-10-17.

COLOFÓN

# Sobre esta colección

Del texto al contexto · Murciélago · Encontrar señales parecidas sin confundirlas con hechos · 2026.

| # | Título | Animal | Ángulo |
|---|---|---|---|
| I | Agentes de IA: De la idea a la producción | Pulpo | Inteligencia distribuida |
| II | Construyendo con Agentes de IA | Hormiga | Enjambre emergente |
| III | Agentes de IA: Una guía ilustrada | Cuervo | Uso inteligente de herramientas |
| IV | Patrones Arquitectónicos para Sistemas Multiagente | Abeja | Patrones de coordinación probados |
| V | Agentes de IA con MCP | Delfín | Comunicación y protocolo |
| VI | Agentes de IA en Acción | Zorro | Acción inteligente y adaptativa |
| VII | Agentes de IA en la práctica | Castor | Ingeniería orientada a producción |
| VIII | El Segundo Cerebro Corporativo | Ardilla | Conocimiento almacenado y recuperable |
| IX | La Federación de Cerebros | Suricata | Vigilancia distribuida |
| X | La Máquina de Predecir | Cachalote | Percepción construida desde señales y patrones |
| **XI** | **Del texto al contexto** | **Murciélago** | **Encontrar señales parecidas sin confundirlas con hechos** |
| — | El Castor y el Plano — Spec-Driven Development con agentes | Castor | Construir siguiendo un plano vivo |